University of Calgary
Faculty of Science
Department of Computer Science

SENG 300 - Fall 2026

Assignment 1: Requirements, Design, and Implementation

Weight: 10% of final grade
Team size: 2 to 7 students
Due: Monday, 19 October 2026 at 11:59 PM
Deliverable: one ZIP file per group plus a shared git repository

1. Overview

This assignment takes you from a client request all the way to working code. You will analyze and model the requirements of a system, design it, and then implement part of that design in Java. The description below explains what the system should do. Your job is to understand how the system is meant to work, model it, design a clean solution, and build part of it.

This is a team assignment. You will also collaborate with **git**, the same way a real development team would, and the quality of how you use git is part of your grade.

2. The client's description

The following description was given to your organization by the client. Read it carefully; it describes what the GoCycle system should do.

> Our city's Mobility Office wants to give residents an easy way to get around without a car. We would like you to build the software for a shared bike and scooter service. We are calling it **GoCycle**.
>
> The idea is simple. We place docking stations around the city. Each station has a number of docking points, and each point can hold one vehicle. A rider walks up to a station, picks a vehicle, takes it, rides it, and returns it to any station that has a free point. Most people will use a phone app, and there is also a small screen and a card reader at each station for people who do not have the app.
>
> We offer three kinds of vehicles: regular pedal bikes, electric bikes, and electric scooters. The electric ones have a battery, and the system does not let someone take an electric vehicle when its battery is below 20 percent.

People can sign up as casual riders who pay for each trip, or as members who pay monthly and get cheaper trips. Students get a reduced rate as well. We expect to add new kinds of riders in the future, so pricing should be easy to extend.

A trip costs an unlock fee plus a charge for every minute. Members pay a lower per-minute rate than casual riders, and students pay a reduced rate too. A trip longer than two hours adds a long-trip surcharge. At the start of a trip the system places a refundable deposit hold, and it releases that hold when the trip ends.

When a rider wants to return a vehicle, they bring it to a station and lock it into a free point. The system makes sure the vehicle is really locked before it ends the trip and stops charging. If the destination station has no free point, the system directs the rider to the nearest station that has one; the trip does not end and charging does not stop until the vehicle is locked at a free point.

Riders can report when something is wrong with a vehicle, like a flat tire or broken brakes. The electric vehicles also report low battery or a fault on their own. When a vehicle is reported as broken, it is taken out of service so nobody else rents it, and our maintenance crew is notified. Our operators, who watch the whole network from a control room, see it as well.

For paying, we accept credit cards, the city transit card, and phone wallet payments. Some neighbourhoods are run as separate zones with their own rules, and a zone may allow only some of these payment methods.

We care a lot about being welcoming to everyone. The screens support more than one language, and they read information aloud for riders who need it. Each zone has its own brand colours and logo that appear on the screen, and we adjust those from configuration.

Operators in the control room see the state of every station and vehicle, move vehicles between stations when one station is too full or too empty, and take a vehicle out of service or put it back. When a trip is finished and paid for, the rider gets a receipt on the app or by email, and the system is ready for the next rider.

If you have ideas to make the service better for our riders, we would be glad to hear them. We look forward to your proposal.

3. What you must produce

Your submission has two parts: a **report** and a **git repository with code**.

Part A - Requirements and design (report)

1. **Requirements classification.** Classify each requirement using the three categories from the lectures: **functional** (what the system must do), **quality** (how well it must work), and **constraint** (a boundary on the solution). Group them so a reader can follow them.

2. **Use case model.** A use case diagram plus use case descriptions for the main scenarios (for example, starting a trip, returning a vehicle, and reporting a fault).
3. **Class diagram.** A UML class diagram for your design, with at least **8 classes**, showing inheritance, composition, and the associations that matter. Your design must apply the **SOLID** principles and must make room for the design patterns you implement in Part B.
4. **Design rationale.** One page explaining how your design supports flexibility and maintainability, and where each SOLID principle and each pattern shows up.

Part B - Implementation (git repository)

5. **Java implementation of your design.** Implement the core of your design in Java. Your implementation must:
    - Apply the **SOLID** principles, and
    - Use **at least two design patterns** (for example Strategy, Observer, Factory Method, Abstract Factory, or Singleton), where each pattern solves a real problem in your system. You may also apply design patterns that were not covered in class.

For each design pattern you use, include in your report: a short problem description, your justification for choosing it, a small UML snippet, and a short paragraph on its benefits and trade-offs.

4. Teamwork and git

- Teams are **2 to 7 students**, formed within your tutorial section. Tell your TA who is on your team. You cannot change teams later in the course.
- You must do the implementation and keep your design in a **shared git repository**.
- Your git history is part of your grade. Use branches, make **small, focused commits**, and write **clear commit messages** that say what changed and why.
- Every team member must contribute, and this must be visible in the commit history. Merge your work together through the project rather than committing only at the end.
- Your repository must contain a `README.md` with build and run instructions and the UCIDs of every member.

5. Submission format

Submit **only one ZIP file per group** to the A1 Dropbox. Do not submit more than once for the same group.

1. The report as a PDF, with a cover page listing every member's name and UCID.

2. A `src/` folder with your Java source code.
3. A `README.md` with build and run instructions and all UCIDs.
4. A file listing every member's UCID and email address. If this file is missing, some marks will be deducted.
5. The link to your git repository and an exported commit log.
6. A list of any outside sources you used, in the submission comments.

6. Grading rubric (10%)

Criterion	Weight
Requirements model (classification, use case diagram and descriptions)	3%
Design (class diagram, SOLID, design rationale)	2%
Java implementation and correct use of at least two patterns	2%
Git usage (branches, small focused commits, clear messages)	1%
Excellence and overall quality (instructor judgment)	**2%**

The excellence mark rewards submissions that go beyond the minimum in clarity, professionalism, and the quality of the design and code. It includes quality across all components of the work. It is 20% of this assignment and is awarded at the instructor's and TAs' judgment.

7. Academic integrity

You must cite all sources of information you use, other than the materials we hand you directly (this description, the course notes, the textbook, and anything posted to the discussion forum). List your sources in your submission comments. Using outside information without citing it is academic misconduct. Using it with a citation is not misconduct but may lower your grade depending on its nature. If you are unsure, ask us first.

8. Late penalties

Submissions made more than 48 hours after the due date receive an F. Otherwise the late penalty is:

```
late_penalty = min { hours_late, 48 } / 48
```

`hours_late` is determined by D2L. Submissions less than an hour late receive no penalty (a one-hour grace period). Deadlines matter; if you ask for an extension, the answer will be "no" except under unusual circumstances.